

Apollo Lunar Module — One-Shot Motion-Planning OCP

Code Guide (`apollo_full.py`)

Overview

`apollo_full.py` is a single self-contained script that plans a soft lunar landing. It builds *one* open-loop optimal control problem (OCP), solves the resulting nonlinear program (NLP) *once*, appends a short analytic engine-cut-off phase, and plots the result. The method is **direct multiple shooting** with an **RK4** integrator, assembled with CasADi's `Opti` modelling layer and solved by **IPOPT**. The end-to-end pipeline is

parameters $\longrightarrow$ `solve_landing` $\longrightarrow$ `cutoff_freefall` $\longrightarrow$ report + plots.

File layout

The file follows six banner-marked sections:

1. **Parameters** — three @dataclasses (`LMPParams`, `OCPConfig`, `Scenario`).
2. **Dynamics** — the continuous model `flat_moon_6dof` and the integrators.
3. **OCP build & solve** — `solve_landing` and the analytic `cutoff_freefall`.
4. **Landing report** — `print_report`, `print_cutoff_report`.
5. **Plots** — states, controls, actuators, thrusters.
6. **Trajectory plot** + `__main__` driver.

1. Parameters

Everything tunable lives in three dataclasses, so no magic numbers are buried in the logic:

- `LMPParams` — the vehicle: mass, inertia J , lunar gravity, DPS thrust limits $[T_{\min}, T_{\max}]$, gimbal limit, the actuator models (thrust first-order lag τ_T , gimbal second-order ω_n, ζ), and the 16-thruster RCS geometry. `rcs_geometry()` returns the 6×16 allocation matrix B mapping thruster forces to a body wrench; `B_rcs` caches it (it is queried thousands of times while the graph is built).
- `OCPConfig` — the problem: horizon N , step `dt`, `integrator`, the path limits, the cost weights (`Qs`, `Qf`, `Rw`, `Rd`), and the cut-off altitude `h_contact`.
- `Scenario` — the initial state `x0` (position, velocity, attitude, rates) and the target `x_target`.

2. State, dynamics and integrators

The rigid body carries 12 states; five *actuator* states are appended so the optimiser sees the real, rate-limited hardware response:

Index	Symbol	Meaning
0–2	x_E, y_E, z_E	surface-frame position ($z_E \leq 0$ above ground)
3–5	u, v, w	body-frame velocity
6–8	ϕ, θ, ψ	Euler angles (3–2–1)
9–11	p, q, r	body rates
12	T	actual thrust (1st-order lag toward T_c)
13–14	$\delta_p, \dot{\delta}_p$	pitch gimbal (2nd-order toward $\delta_{p,c}$)
15–16	$\delta_y, \dot{\delta}_y$	yaw gimbal (2nd-order toward $\delta_{y,c}$)

`flat_moon_6dof(x,u,lm)` returns $\dot{x}$: navigation ($\dot{r} = C_{E/B}v$), body-frame translation (gravity + gimballed thrust + RCS force), Euler kinematics, Euler’s rotational equations, and the actuator ODEs. `rk4_step` advances one step with classical RK4; `rk2_step` (Heun) is a cheaper alternative, and `INTEGRATORS` is the `{‘rk4’, ‘rk2’}` registry `solve_landing` selects from.

3. Building and solving the OCP (`solve_landing`)

The decision variables are the full state trajectory $X \in \mathbb{R}^{17 \times (N+1)}$ and the commands $U \in \mathbb{R}^{19 \times N}$. The constraints and objective are assembled on `Opti`:

- **Initial condition** $X_{:,0} = x_0$.
- **Dynamics (equality)** — multiple-shooting defects $X_{:,k+1} = \Phi_{\text{RK4}}(X_{:,k}, U_{:,k})$ for every interval.
- **Path bounds** — altitude, velocity, attitude, rate, thrust and gimbal limits on the states, and thrust/gimbal/RCS limits on the commands.
- **Cost** — a stage term (state error Q_s + command effort R), a *control-rate* term $R_\Delta \|\Delta u_k\|^2$ that damps gimbal chatter, and a terminal term Q_f pulling toward the contact condition ($z_E = -h_{\text{contact}}$).
- **Warm start** — states linearly interpolated $x_0 \rightarrow \text{target}$, thrust held at hover; a good guess cuts IPOPT iterations.

IPOPT runs with the **exact Hessian** (fewer, more informative iterations) and `expand=True`, which rewrites the problem graph from MX to SX so the derivatives evaluate about twice as fast. The `acceptable_*` tolerances let it stop at a good feasible point instead of grinding to the tight tol. The function returns the solved `(Xs, Us)`.

4. Engine cut-off and touchdown (`cutoff_freefall`)

The powered OCP deliberately ends a metre or so up, at the *contact* altitude. `cutoff_freefall` then models the real DPS shutdown: it hard-cuts the thrust ($T := 0$, gimbals zeroed, RCS off) and integrates the same 6-DOF dynamics with fine 0.02s steps until $z_E = 0$, interpolating the final sub-step so touchdown sits exactly on the ground. For a near-rest contact the touchdown speed is $v_{\text{td}} = \sqrt{w_c^2 + 2gh_{\text{contact}}}$. This keeps the OCP well-posed near $z = 0$ (no thrust gating, no hovering pathology).

5. Output

`print_report` summarises the powered terminal state (position, speed, attitude vs. success thresholds); `print_cutoff_report` shows the cut-off event and the analytic touchdown. Five plot helpers save PNGs: `plot_states`, `plot_controls` (DPS + net RCS wrench), `plot_actuators` (commanded vs. actual), `plot_thrusters` (16 jets), and `plot_trajectory_with_axes` (3-D path with body-axis triads; the free-fall segment is stitched on via `np.hstack([Xs, Xff])`).

Tuning & performance knobs

Knob	Effect	Trade-off
<code>N</code> , <code>dt</code>	grid size / horizon	smaller $\Rightarrow$ faster solve, coarser near ground
<code>integrator</code>	<code>rk4</code> / <code>rk2</code>	<code>rk2</code> cheaper/step but often more iterations
<code>expand</code>	SX graph	$\sim 2\times$ faster derivatives (kept on)
<code>h_contact</code>	cut-off altitude	lower $\Rightarrow$ softer touchdown $\sqrt{2gh}$
<code>Rd[gimbal]</code>	move-suppression	higher $\Rightarrow$ smoother gimbals, stiffer NLP
<code>Qf</code>	terminal accuracy	higher $\Rightarrow$ tighter landing, harder solve

The dominant cost is IPOPT's KKT factorisation times the iteration count, so `N` is the biggest speed lever; the trajectory becomes feasible early and the remaining iterations polish optimality on a poorly-scaled problem.